1 Data, Databases, and DBMS Fundamentals

Why this matters: Every app that remembers an order, mark, booking, or payment must make sure its data remains useful and trustworthy. This chapter explains why a database is more than a collection of files.

Learning objectives: By the end, you should be able to distinguish data and information; define database, DBMS and database system; identify major DBMS services; explain the weaknesses of isolated files; identify users and responsibilities; and read a small working CampusDB query.

Prerequisites: None. Familiarity with files, folders and simple arithmetic is enough. Later links: Ch. 2 (architecture), Ch. 4 (ER), Ch. 5 (keys), Ch. 7 (SQL), Ch. 10 (transactions).

Concept

The one-sentence model: Users and applications ask a DBMS to work with stored data, and the DBMS enforces configured rules while coordinating access, storage and recovery. *A DBMS cannot guarantee that an application's business rules are correct if those rules were never specified.*

1.1 Begin with a familiar example

Imagine a university portal. A learner can see enrolled courses; a lecturer can upload marks; the accounts office can record fees. These screens do not need separate copies of every student fact. Instead, a shared database stores a student's identity once and connects it to academic and financial records under defined permissions.

Suppose student 101 changes city from Patna to Delhi. In an unmanaged collection of files, the admission office may update its spreadsheet while the hostel and examination offices keep the old city. A database design can store one authoritative city value for all three applications to read; controlled access and suitable application logic then make the update consistent.

Real-life use

A college portal illustrates four different questions: **What is stored?** Student and Course facts. **Who may change it?** Authorized staff. **Which relationships are valid?** An enrollment must refer to an existing student and course. **What if two requests arrive together?** The system must coordinate them. The last three are reasons database *management* matters.

1.2 Data versus information

Plain language. Data consists of recorded facts: numbers, names, dates, readings and codes. Information is an interpretation of data in context, useful for answering a question or making a decision. Data need not be

meaningless; the distinction describes how the facts are used.

Formal view. A value is data when it represents a fact in a domain. Processing, organization and context may transform it into information for a particular user. The same value can serve as data in one process and information in another.

Worked Example 1.1: Four examination marks

Input data: 78, 82, 91, 89. The teacher asks for the class average and for how many marks are at least 80.

1. **Organize:** Four marks, all on a 100-point scale.
2. **Compute:** Sum $= 78 + 82 + 91 + 89 = 340$.
3. **Average:** $340/4 = 85$.
4. **Count marks at least 80:** 82, 91 and 89, so the count is 3.
5. **Interpret:** The group average is 85/100; 3 out of 4 learners scored at least 80.

Result: Raw marks became answers to two questions. Note that a high average alone does not establish whether an exam was fair or difficult.

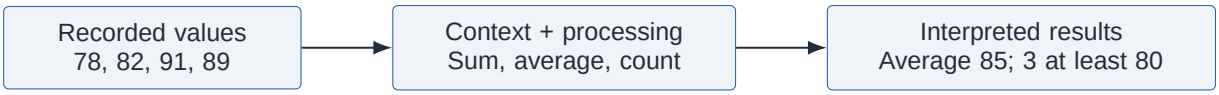


Figure 1.1. Recorded marks become useful information after a question, calculation and interpretation are supplied.

Common mistake

Incorrect: "Data is always unprocessed, and information is always correct." **Correction:** A previously computed average is still data to another process; information can be misleading if the input is incomplete or the calculation is wrong.

1.3 What exactly is a database?

Plain language. A database is an organized collection of related data, such as student records and enrollments, intended to support reliable retrieval and updates.

Formal definition. A database is a persistent, logically related collection of data representing some part of the real world (its *universe of discourse*), organized for access and management. It may be relational, document-based, graph-based, key-value or another model.

A phone contact list can be considered a simple database in the everyday sense. At a technical level, the phone may implement it using a full embedded database, structured application files or a service. Calling any collection of records a database does *not* prove that it has all the services of an RDBMS.

Illustration: A minimal campus database

Instead of writing "Asha – CSE – Patna" as an unstructured line, imagine the following relations. **Meaning:** `StudentID` is the student identifier. `DeptID` links a student to a department when assigned. Student and Department are intentionally separate so "CSE" does not need to be repeated for every student. We will formally justify keys and constraints in Chapter 5.

Table 1.1. Department data in the CampusDB teaching dataset (version 1).

DeptID	DeptName
10	CSE
20	ECE
30	MBA

Table 1.2. Student data in CampusDB version 1. NULL denotes an unassigned department, not an actual department.

StudentID	Name	City	DeptID	CGPA
1	Asha	Patna	10	8.42
2	Ravi	Delhi	10	7.25
3	Meera	Patna	20	9.01
4	Kabir	Jaipur	NULL	6.80

1.4 What is a DBMS? What is a database system?

Plain language. A **database management system (DBMS)** is software that helps people and programs define, store, retrieve, update and protect data. PostgreSQL, MySQL, Oracle Database and SQLite are examples of relational database management systems.

Formal definition. A DBMS provides an interface for describing data structures and performing controlled operations against a database. Depending on the product and configuration, it also provides integrity checks, transactions, concurrent access, authorization, logging and recovery.

Keep three terms separate:

- **Database:** the actual organized data and associated structures.
- **DBMS:** the software that manages database access and behavior.
- **Database system:** DBMS plus stored data, applications, procedures, users and computing resources.

Worked Example 1.2: Find every student from Patna

A user asks: “Which students currently have City = Patna?” The request goes from the portal to the DBMS; the DBMS reads the relevant stored data and returns matching rows. For the CampusDB data above, the answer is **Asha and Meera**. It does not need to return the entire student file to the portal.

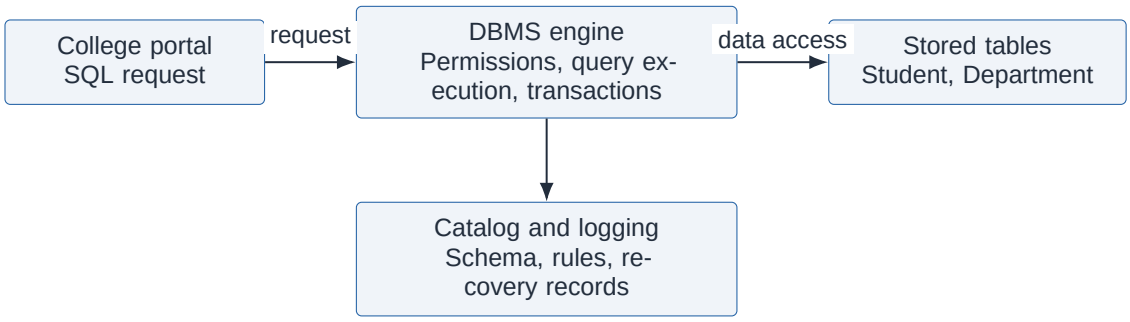


Figure 1.2. A simplified DBMS ecosystem. Storage, catalog and logging are shown conceptually; real implementations organize these components differently.

Exam checkpoint

A database is *data*; a DBMS is *software*; a database system includes the *data, DBMS, applications and people*. These terms are related but not interchangeable.

1.5 Why ordinary files become difficult

A CSV file or spreadsheet is useful for a short list. Problems arise when separate programs keep copies of shared facts and must coordinate many updates. File systems provide their own permissions and durability features, so it would be inaccurate to claim that ordinary files have *no* security or reliability. The key advantage of a DBMS is a set of data-aware features, rather than merely saving bytes.

Worked Example 1.3: One address in three files

Suppose three college offices separately store the same student's city.

File	Recorded value for StudentID 1	Status
admissions.csv	Asha, Patna	old
hostel.csv	Asha, Delhi	updated
exam.csv	Asha, Patna	old

Asha has moved to Delhi. Updating only `hostel.csv` leaves two outdated copies. This is an **update anomaly**. A normalized shared design stores the authoritative city in one student record; each authorized application reads it from there.

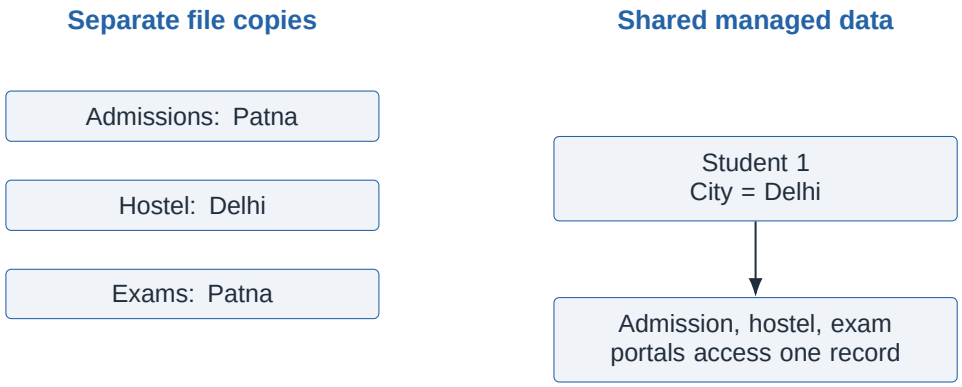


Figure 1.3. Inconsistent file copies versus one authoritative record. Actual deployments may cache or replicate data and still require consistency controls.

The main file-processing problems

Problem	Simple symptom	DBMS feature that can help
Redundancy	City repeated in three independent files	Shared schema and deliberate normalization
Inconsistency	Different offices show different cities	Central updates and integrity rules
Hard retrieval	Write a special script for each report	SQL queries and indexes
Weak data-level access	Staff can open a whole sensitive file	Roles, grants and selected views
Update conflicts	Two people overwrite the same record	Transactions and concurrency control
Recovery work	A process stops halfway through updates	Logging, transactions and backups
Program dependence	A change in file format breaks scripts	Logical schemas and stable interfaces
Invalid references	Enrollment mentions a nonexistent course	Foreign-key constraints

Common mistake

Incorrect: "Using a DBMS removes all duplicate data and automatically prevents all errors." **Correction:** Databases can contain deliberate duplication, inconsistent application rules, incorrect input or missing constraints. Good schema design and operational procedures are still necessary.

1.6 What a DBMS actually does

Five groups of services are worth learning before SQL.

1. **Structure and metadata.** Define tables, column types, relationships and indexes; store their descriptions in a catalog.
2. **Data operations.** Insert, update, delete and search for records, usually through a query language or API.
3. **Rules and permissions.** Check configured constraints and grant appropriate operations to each role.
4. **Concurrent work.** Coordinate users whose transactions overlap; offer defined isolation behavior.
5. **Storage and resilience.** Organize pages and buffers, record recoverable changes and support backups or recovery tooling.

A **query optimizer** also chooses an execution strategy for eligible queries. For example, it may choose an index lookup over a full-table scan if its cost estimates justify that choice. An index is not always faster for every query.

Illustration: Integrity checks on a new enrollment

Suppose an enrollment request says `StudentID = 999` while the database only contains students 1–4. If the Enrollment table declares an appropriate foreign key, the DBMS rejects that reference. Without the constraint, the DBMS might accept it. **The important distinction is that rules must be declared.**

Worked Example 1.4: The last available seat

A university has one free seat in a course section; two students click *Enroll* almost simultaneously.

1. **Without coordination:** both requests read "one seat available"; each adds an enrollment; capacity is exceeded.
2. **Desired invariant:** number of active enrollments $\leq$ section capacity.
3. **Database-supported design:** perform the capacity check and enrollment within a safe transaction strategy, such as locking the section row or using an atomic reservation counter update.
4. **Verify:** after both attempts, one succeeds and one is rejected or safely retried. Which one succeeds depends on scheduling; the system must preserve capacity in either case.

Do not infer that merely using `BEGIN` and `COMMIT` makes any check-then-insert algorithm safe. Chapter 12 derives the correct isolation and locking details.

GATE tip

Typical exam distinctions: **redundancy** means repeated facts; **inconsistency** means conflicting values; **integrity** means satisfaction of defined rules; **isolation** concerns concurrent transactions. A question may describe one while testing another.

1.7 DBMS versus RDBMS

DBMS is the general category. An **RDBMS** is a DBMS based primarily on the relational model, whose logical data consists of relations (commonly shown as tables). SQL is the widely used language of relational systems; implementations vary in their supported syntax and features.

	DBMS (general)	RDBMS (specific)
Logical model	Relational, document, graph, key-value and others	Relations (tables) and relational operations
Linking facts	Depends on the chosen model	Keys and joins; declared foreign keys where used
Examples	MongoDB, Redis and relational systems	PostgreSQL, MySQL, Oracle Database, SQLite
Queries	Product-specific language or API	Usually SQL

A useful counterexample: MongoDB stores documents organized into collections; it is a DBMS but not a traditional relational DBMS. SQLite is an embedded relational DBMS; it does not require a separate database server process for ordinary application use.

Exam checkpoint

Every RDBMS is a DBMS, but not every DBMS is relational. "RDBMS requires that every table have an explicit foreign key" is *not* the definition; relations can exist without any foreign-key relationship.

1.8 Who works with a database?

A **database administrator (DBA)** manages database availability, permissions, performance, backups, restoration and monitoring. A **database designer** specifies the data structures and constraints. An **application developer** writes code and SQL, a **data analyst** builds reports, and an **end user** interacts with the system through a portal or app.

Illustration: University portal responsibilities

Role	Real task
Student (end user)	Read own enrollment and marks
Teacher (end user)	Enter marks only for permitted sections
Developer	Build the course-registration application and parameterized queries
Designer	Define Student, Course, Section and Enrollment rules
DBA	Grant privileges, monitor storage and test backups
Analyst	Produce department-wise enrollment totals

Important: A single person can perform multiple roles in a small project. In larger organizations, duties are often divided for reliability and security.

1.9 Schema, record, metadata and the catalog

Schema means the structural definition, such as `Student(StudentID, Name, DeptID, City, CGPA)`. A **record** is one stored row, such as `(1, Asha, 10, Patna, 8.42)`. **Metadata** describes data: column names, types, constraints and indexes. A **system catalog** stores database metadata so the DBMS and its users can inspect it.

Mini example: If someone changes Asha's city, the *records* change, but the table's *schema* can remain the same. Adding a new `Phone` column changes the schema. Chapter 2 will formalize the distinction between *schema* and *instance*.

SQL lab

PostgreSQL and MySQL (with an appropriate schema selected): the standard `information_schema.columns` view can reveal table structure. Oracle also provides data-dictionary views such as `USER_TAB_COLUMNS`; catalog identifiers and filtering differ across systems. We will learn the catalog in more depth after writing our first tables.

```sql
-- PostgreSQL: inspect public.Student table columns
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_schema = 'public'
  AND table_name = 'student'
ORDER BY ordinal_position;
```

When to run: after creating the table in the next SQL lab. The catalog stores descriptions, not all student rows.

1.10 Our running example: CampusDB version 1

Every SQL lab from this chapter onward will return to **CampusDB**. The stable first fixture has three departments and four students, matching the tables shown earlier. Later chapters explicitly extend it with Teacher, Course, Section and Enrollment. For now, *one unassigned student is allowed*: Kabir's `DeptID` is NULL. A later case study may impose stricter admission rules, but that requirement must be stated rather than silently assumed.

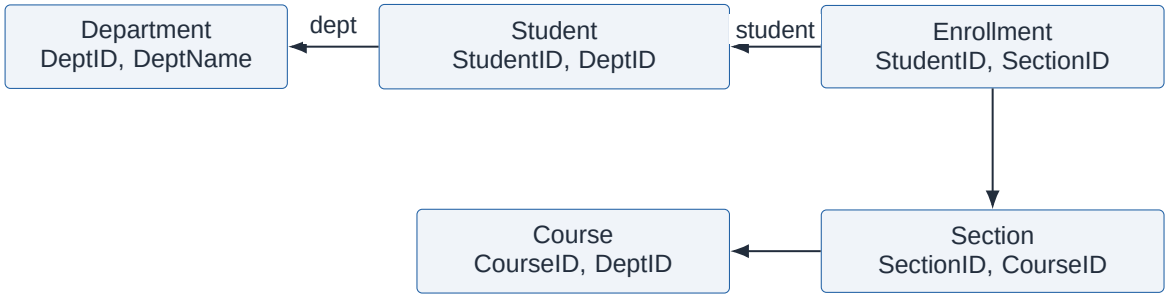


Figure 1.4. Conceptual links in the expanding CampusDB case. The figure is illustrative, not the final complete ER diagram; formal cardinalities and mapping appear in Chapter 4.

SQL lab: Create and query the first two tables

Run the following in a **scratch PostgreSQL database**, not in a live database with valuable tables. To keep the lab repeatable, use the exact inserts below once on a new setup. The full script is also provided as `sql/CampusDB_v1_postgresql.sql`.

```
CREATE TABLE Department (
  DeptID INT PRIMARY KEY,
  DeptName VARCHAR(50) UNIQUE NOT NULL
);

CREATE TABLE Student (
  StudentID INT PRIMARY KEY,
  Name VARCHAR(80) NOT NULL,
  DeptID INT REFERENCES Department(DeptID),
  City VARCHAR(60),
  CGPA DECIMAL(3,2)
);

INSERT INTO Department VALUES
  (10, 'CSE'), (20, 'ECE'), (30, 'MBA');

INSERT INTO Student VALUES
  (1, 'Asha', 10, 'Patna', 8.42),
  (2, 'Ravi', 10, 'Delhi', 7.25),
  (3, 'Meera', 20, 'Patna', 9.01),
  (4, 'Kabir', NULL, 'Jaipur', 6.80);

SELECT StudentID, Name
FROM Student
WHERE City = 'Patna'
ORDER BY StudentID;
```

Expected query output:

StudentID	Name
1	Asha
3	Meera

Explain the output step by step: (1) `FROM` chooses the Student table. (2) `WHERE` keeps Asha and Meera because their city is Patna. (3) `SELECT` displays only the identifier and name. (4) `ORDER BY` sorts the result by identifier.

We will study each clause fully in Chapter 7.

Common mistake

Do not run `CREATE TABLE` repeatedly in the same schema without checking whether the table exists. A second run can fail with "relation already exists." Use a new scratch database, or reset the lab intentionally. Avoid running `DROP TABLE` on production data.

1.11 When would you choose files instead?

A DBMS has benefits, but also requires learning, maintenance, resource use and an appropriate setup. A single, read-only configuration file may not need a database. A multi-user course registration service typically does. The relevant question is *which storage approach meets the requirements?*

Case	Reasonable starting choice
Personal static reading list	CSV or a simple local file may be sufficient
Mobile app with searchable offline records	Embedded relational DBMS such as SQLite may be convenient
College enrollment across many users	Server-based transactional RDBMS with defined constraints
Specialized relationship network	Graph DBMS may fit, depending on queries and operational needs

Real-life use

Modern-system snapshot (products vary): PostgreSQL is an open-source relational DBMS with a SQL interface and documented transactions. MySQL's InnoDB engine provides transactional storage and multi-versioning. MongoDB is a document DBMS. These are implementation examples, not definitions of the underlying concepts. Official documentation links appear at this chapter's end.

1.12 What Chapter 1 does not yet teach

You may now have encountered words such as *foreign key*, *transaction*, *index* or *catalog*. Memorizing a shallow definition is not the goal. In subsequent chapters we will derive the specific rules, build schemas and test query outputs. For today, make sure you can explain *why* structured shared data and controlled operations matter.

Chapter summary

- Data becomes information when a question and context give recorded facts meaning.
- A database stores related facts; a DBMS manages access and operations; a database system includes the surrounding people and applications.
- Isolated file copies may create redundancy, inconsistency and hard-to-coordinate updates.
- A DBMS offers structure, queries, integrity checks, authorization, concurrency control and recovery tools; configured rules and good application code still matter.
- An RDBMS is a kind of DBMS that uses the relational model.
- CampusDB version 1 contains Department and Student tables; its one unassigned student's department is explicitly nullable.

One-page cheat sheet

CHAPTER 1 | QUICK RECALL

Definitions: **Data** = recorded facts. **Information** = interpreted facts in context. **Database** = organized, related stored data. **DBMS** = software for controlling access and operations. **Database system** = DBMS + data + people + apps + environment.

File problem vs DBMS help:

Repeated facts → update anomalies	Shared schema / normalization
Invalid references	Declared keys and constraints
Many simultaneous updates	Transactions and concurrency controls
Difficult searches	Declarative queries; suitable indexes
Confidential fields	Privileges and filtered views
Crash during update	Recovery and logging facilities

RDBMS $\subset$ DBMS. PostgreSQL, MySQL, Oracle Database and SQLite are relational. MongoDB is document-oriented.

Two data examples: $78 + 82 + 91 + 89 = 340$; mean $= 85$. For CampusDB, Patna students are Asha and Meera.

Six college roles: DBA, designer, developer, analyst, teacher and student.

Exam traps: (i) DBMS $\neq$ database; (ii) DBMS $\not\Rightarrow$ error-free data; (iii) data need not be meaningless; (iv) storing tables alone does not prove an RDBMS; (v) schema $\neq$ a single stored row.

What follows: Ch. 2 explains logical vs physical data independence; Ch. 4 designs entities; Ch. 5 establishes keys; Ch. 7 introduces SQL; Ch. 10 explains ACID.

1.13 Chapter-end assessment

Difficulty tags: E = Easy, M = Medium, H = Hard. All solutions and model answers are reserved for the separate `answers/ch01_answers.tex` book-end key. SQL lab expected tables are shown so students can verify their own query results.

A. Multiple-choice questions (10; one answer each)

MCQ 1.1 **[Easy]** Which statement best defines a DBMS? (A) Every collection of files (B) Software that manages database operations (C) Only a query result (D) A network protocol.

MCQ 1.2 **[Easy]** Which pair is correct? (A) Schema = one row (B) Metadata = column definitions (C) Tuple = DBMS software (D) DBA = backup file.

MCQ 1.3 **[Easy]** Which is a *kind* of DBMS? (A) RDBMS (B) CSV extension (C) Row number (D) Monitor.

MCQ 1.4 **[Easy]** The average of 78, 82, 91 and 89 is (A) 84 (B) 85 (C) 86 (D) 88.

MCQ 1.5 **[Medium]** When three departments store different copies of the same address, the most direct symptom is (A) data independence (B) guaranteed atomicity (C) inconsistency (D) indexing.

MCQ 1.6 **[Medium]** Which DBMS mechanism may help prohibit an enrollment for a nonexistent student? (A) Foreign key (B) Sorting (C) Screen resolution (D) File compression.

MCQ 1.7 **[Medium]** What best describes the catalog? (A) All student marks only (B) A table of application icons (C) Metadata describing database objects (D) A backup schedule only.

MCQ 1.8 **[Medium]** A task updates two related tables and must behave atomically. What is most relevant? (A) Transaction (B) Font selection (C) CSV delimiter (D) Data chart.

MCQ 1.9 **[Hard]** Which claim is *not* generally true? (A) RDBMS is a DBMS (B) Relational data is logically organized into tables (C) Every DBMS uses SQL and foreign keys (D) DBMS products can differ.

MCQ 1.10 **[Hard]** Two requests read the final seat simultaneously and both insert an enrollment. What was missing? (A) Proper concurrency-safe capacity enforcement (B) More duplicate spreadsheet copies (C) Longer table names (D) Another user-interface color.

B. Short answers (5; 2–3 marks)

SA 1.1 **[Easy]** Define data and information; give one real example that distinguishes them.

SA 1.2 **[Easy]** Differentiate database, DBMS and database system in three sentences.

SA 1.3 **[Medium]** Explain redundancy and inconsistency using a hostel-office example.

SA 1.4 **[Medium]** Name four functions of a DBMS and one associated feature for each.

SA 1.5 **[Medium]** Define metadata and system catalog. Why would a developer inspect them?

C. Long answers (3; 5–10 marks)

LA 1.1 **[Medium]** Explain at least six limitations of isolated file processing and relate each to an appropriate DBMS facility. Include a diagram.

LA 1.2 **[Medium]** Compare DBMS and RDBMS. Discuss logical tables, keys, SQL and at least three different database products without claiming that every DBMS is relational.

LA 1.3 **[Hard]** Design a database-backed college-portal workflow. Identify roles, stored entities, rules, transaction needs and one rule a DBMS cannot invent automatically.

D. Numerical and practical problems (4)

P 1.1 **[Easy]** A lab records marks $78, 82, 91, 89$. Calculate the sum, mean and number of values ≥ 80. Interpret each result.

P 1.2 **[Medium]** Three offices each store 250 student rows. (i) How many stored rows exist in all copies? (ii) If a student's city is updated in just one copy, how many copies remain stale? (iii) What design change can remove this avoidable duplication?

P 1.3 **[Medium]** A simple fixed-record file has 4,000 records and fits 20 records per page, with no page overhead or fragmentation. Calculate pages required and page reads for a full sequential scan, assuming no pages are cached.

P 1.4 **[Hard]** In CampusDB, Department contains IDs 10, 20, 30, while Student contains IDs 1–4. Assess these attempted Student inserts separately with all other required columns valid: (i) ID 5, DeptID 20; (ii) ID 2, DeptID 30; (iii) ID 6, DeptID 40; (iv) ID 7, DeptID NULL. State which constraints matter.

E. GATE-style questions (5)

G 1.1 **[MCQ, Easy]** Which is *primarily metadata*? (A) A particular learner's CGPA (B) A column's declared data type (C) A submitted mark (D) An enrollment timestamp.

G 1.2 **[MCQ, Medium]** Which database constraint category is most directly related to disallowing a reference to a nonexistent department? (A) Referential integrity (B) Sorting (C) Data compression (D) Data independence.

G 1.3 **[MSQ, Medium]** Choose all database-management responsibilities: (A) Access authorization (B) Recovery support (C) Predicting every business rule (D) Query processing.

G 1.4 **[MSQ, Hard]** Which claims are true? (A) SQLite is relational (B) All databases are relational (C) A DBMS can have an internal catalog (D) A DBMS guarantees no application bugs.

G 1.5 **[NAT, Medium]** A file organization keeps three copies of each of 1,200 records. After centralization it keeps one copy of each, ignoring all other overhead and redundancy. What percentage of the original record copies are eliminated? Enter a percentage rounded to two decimals.

F. Viva / interview prompts (5)

V 1.1 **[Easy]** Give a 20-second explanation of a DBMS to a first-year student.

V 1.2 [Easy] Why is a spreadsheet not automatically equivalent to a fully configured RDBMS?

V 1.3 [Medium] A student changed their address, but two offices show old values. What went wrong?

V 1.4 [Medium] Who is responsible for backups and permissions in a typical production database setup?

V 1.5 [Hard] Is a database constraint sufficient to stop every double-booking race? Explain briefly.

G. Hands-on SQL lab exercises (2)

These exercises use the displayed CampusDB seed tables. Put your SQL in a file, run it, and compare with the expected result. Complete SQL solutions are in the separate key.

LAB 1.1 [Easy] Return `StudentID, Name` for students in Patna, sorted by `StudentID` ascending. **Expected rows:** $(1, \text{Asha}), (3, \text{Meera})$.

LAB 1.2 [Medium] Return one row with columns `TotalStudents` and `AssignedStudents`; the second count excludes NULL departments. **Expected row:** $(4, 3)$.

H. Spot the mistake (2)

SM 1.1 [Easy] "A DBMS is simply the set of rows stored on disk. PostgreSQL is an example of a database, not of database software." Identify and correct two errors.

SM 1.2 [Medium] "Because CampusDB uses a relational DBMS, any invalid `DeptID` must be accepted as long as the `StudentID` is new." Explain the mistake, referring to the schema and its declared rules.

Notes and revision practice

Active recall: Close the book and draw Figure 1.2 from memory. Then explain the difference between a DBMS capability and a rule that must be configured. **Your notes:**

References for product-specific examples

- PostgreSQL, *Official Tutorial*: `https://www.postgresql.org/docs/current/tutorial.html` (retrieved 3 November 2026).
- MySQL, *8.4 Reference Manual, InnoDB Transaction Management*: Official MySQL documentation (retrieved 3 November 2026).
- MongoDB, *Data Modeling*: Official MongoDB documentation (product-specific overview; verify version when implementing).